Математическая логика и теория алгоритмов

Двоичные функции

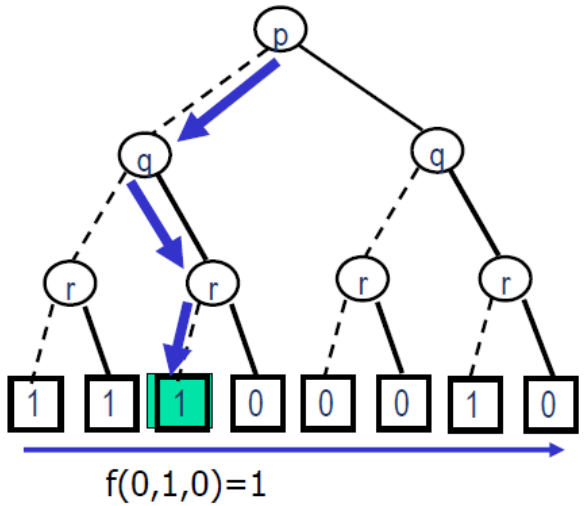
Binary Decision Diagrams (BDD) Бинарные решающие диаграммы

Различные представления одной и той же булевой функции

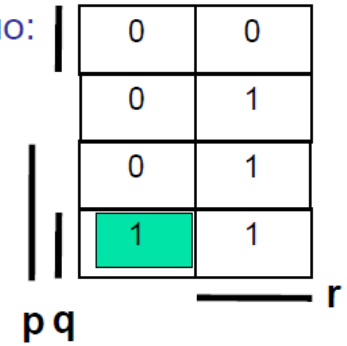
1. Таблица истинности:

pqr	f
000	1
001	1
010	1
011	0
100	0
101	0
110	1
111	0

2.Семантическое дерево:



3.Карта Карно:



- Произвольная формула, ДНФ и КНФ не являются каноническими представлениями
- ТИ, семантическое дерево, Карта Карно, СДНФ, СКНФ, полином Жегалкина являются каноническими представлениями

4. Логическая формула f(p,q,r) = ¬p∨q⊕rq(p∨r)

5. СДНФ: f = ¬p ¬q r ∨ ¬p¬q¬r ∨ q p¬r ∨ q ¬p¬r

6. ДНФ: f = ¬p ¬q ∨ q ¬r

7. КНФ: f = (¬p∨q) (¬q∨¬r),

8. СКНФ: f = (¬p∨q∨r)(p∨q∨¬r)(¬q∨¬r∨p) (¬q∨¬r∨¬p)

9. Полином Жегалкина: f(p,q,r) = 1 ⊕p ⊕pq ⊕qr

Но все они громоздки!!
Сложность $O(2^n)$

Как вычислить значение булевой функции?

- Как вычислить значение БФ по значению ее аргументов?

Нет ничего проще! Берем таблицу истинности и смотрим соответствующую строку

pqr	f
000	1
001	1
010	1
011	0
100	0
101	0
110	1
111	0

Проблема:

ТИ булевой функции от 50 аргументов имеет 2^{50} строк, (миллион миллиардов строк!). Как ее хранить, как с ней работать?

Будем функцию представлять формулой, например, СДНФ

$f = \neg p \neg q\, r \vee \neg p \neg q \neg r \vee q\, p \neg r \vee q \neg p \neg r$

Подставляем значения переменных в конъюнкты и вычисляем!

Проблема:

СДНФ булевой функции от 50 аргументов имеет порядка 10^{17} конъюнктов! (сотни миллионов миллиардов конъюнкций). В них нужно подставить значения переменных, чтобы вычислить значение функции. При скорости 10000 конъюнктов в секунду вычислять 100 000 лет

Проблема зависимости двоичной функции от конкретной переменной

- Проблема зависимости двоичной функции от конкретной переменной – классическая проблема теории двоичных функций

$F = \bar{x}1\bar{x}2\bar{x}3\bar{x}4\bar{x}5 \vee \bar{x}1\bar{x}2\bar{x}3\bar{x}4\bar{x}5 \vee \ldots \vee x1x2x3\bar{x}4x5 \vee \bar{x}1x2x3x4x5$

Все переменные присутствуют в ДНФ функции F

Но от каких переменных функция действительно зависит?

Как узнать?

По СДНФ?

По ДНФ?

По ТИ?

ЧТО ДЕЛАТЬ?

ТИ функции от 20 переменных имеет $\sim 2^{20}$ ($\sim 10^6$)

МИЛЛИОН строк

Проблемы, связанные с БФ

Для заданной булевой функции f, представленной в некоторой форме:

- Найти значение f на конкретной интерпретации (наборе значений аргументов)
- Найти все интерпретации, на которых f = 1
- По функциям f1 и f2 построить функции $\neg f1$, $f1 \otimes f2$ (где $\otimes = \wedge, \vee, \Rightarrow \oplus$, ...)
- Проверить, является ли булева функция f
  - выполнимой (есть ли наборы, на которых f равна 1)
  - общезначимой (на всех ли наборах f равна 1)
  - невыполнимой (на всех ли наборах f равна 0)
- Проверить, совпадают ли две функции F1 и F2 (= невыполнимость $F1 \oplus F2$)

Работать с БФ удобно и легко, если число переменных 2-5!

Как задачи с БФ решаются при большом числе переменных??

Н И К А К!!

Все проблемы для БФ при большом числе переменных классическими методами не решаются

Немасштабируемость решения задач с БФ

Масштабируемость = scalability

БФ, представленные стандартными способами, не позволяют эффективно решать проблемы вследствие экспоненциального роста представления БФ

Это - отражение главного положения теории сложности вычислений:

ЕСЛИ время, требуемое для решения конкретных экземпляров задачи, растет экспоненциально с увеличением размеров этих экземпляров,

ТО эта задача является практически неразрешимой,

ЧТО ДЕЛАТЬ?

СДНФ функции от 50 переменных имеет ~2^{50} (~ 10^{15})
МИЛЛИОН МИЛЛИАРДОВ конъюнктов,
нужно оперировать с терабайтами информации

Если перебирать по 1000 конъюнктов в секунду, для поиска нужно
больше 30 000 лет (~ 10^{12} сек, ~3×10^{8} час ~10^{7} дней ~ 3×10^{4} лет)

BDD – ФОРМА ПРЕДСТАВЛЕНИЯ БУЛЕВЫХ ФУНКЦИЙ

В 1986 г. Randal Briant предложил новую эффективную форму представления БФ, Binary Decision Diagram (BDD) – Бинарные решающие диаграммы

R.E.Bryant. Graph-based algorithms for boolean functions manipulation. *IEEE Transactios on Computers, 8, 1986 (CMU) – самая цитируемая работа в Computer Science*

BDD имеют множество хороших свойств, с их помощью в настоящее время решаются многие задачи.
Основные свойства: BDD –
- эффективное представление БФ
- для любой БФ представление BDD является КАНОНИЧЕСКИМ (единственным)

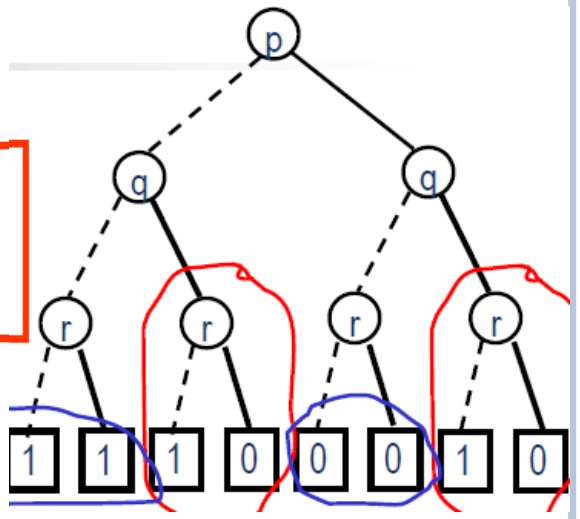
Представление в BDD используемых на практике двоичных функций растет полиномиально, или даже линейно от числа двоичных переменных, хотя существуют функции, сложность представления которых экспоненциальна
Наиболее экономное представление в BDD там, где есть симметрия (например, схемы и аппаратные системы)

Бинарные решающие диаграммы (Binary Decision Diagrams)

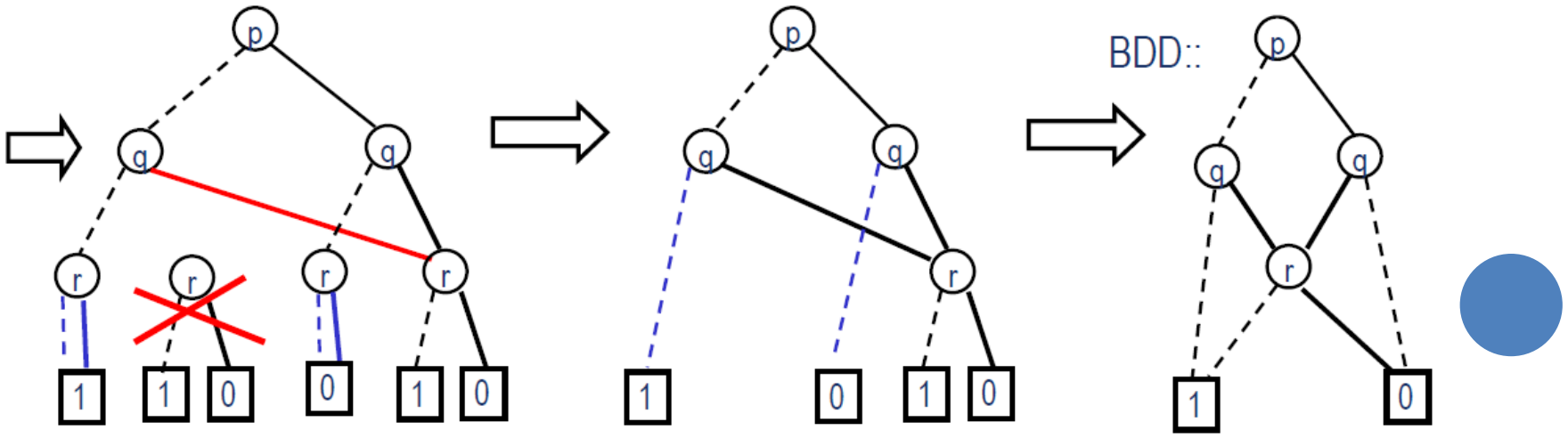
BDD-это семантическое дерево, НО без избыточностей
Корневые и промежуточные вершины помечены переменными, из них выходят ровно два ребра

Binary – потому что двоичные переменные

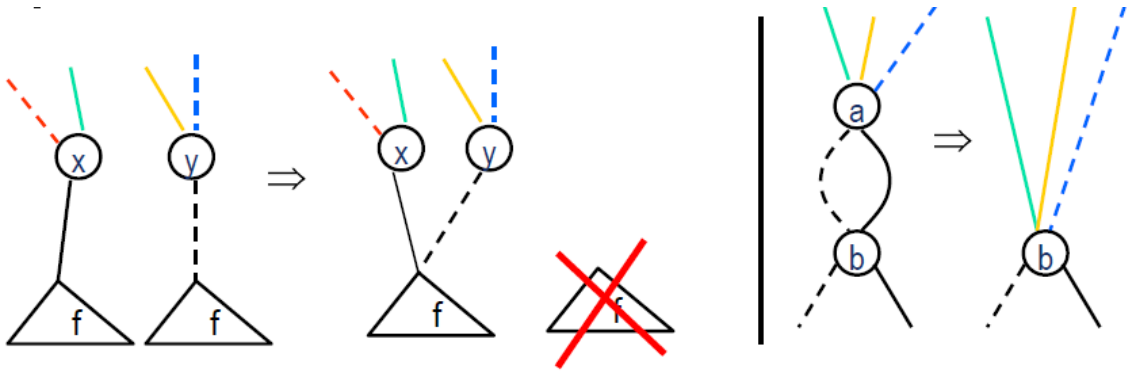
Decision – потому что решения принимаются при направленном движении по графу (диаграмме)



Семантическое дерево:



Алгоритм Reduce: преобразование семантического дерева в BDD



C1: Если в графе имеются одинаковые подструктуры, то оставляем только одну из них. Поиск таких структур можно выполнять от листьев вверх по ярусам графа

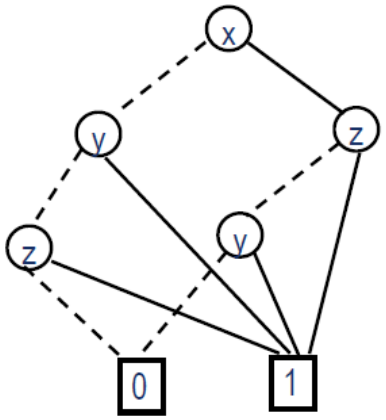
C2: Если обе выходные дуги вершины v ведут в одну вершину, то вершина v выбрасывается

Повторное применение C1, C2 в любом порядке к семантическому дереву функции f или к любому полученному из него графу, приводит к минимальному представлению f в виде ациклического графа, которое называется BDD

Вопрос: почему алгоритм Reduce завершается??

BDD, OBDD и ROBDD

Не любая BDD является каноническим представлением, нужно еще ограничение на порядок переменных



BDD, но не OBDD

Ordered BDD - это BDD, в которой по каждому пути переменные встречаются в КАКОМ-ТО ОДНОМ фиксированном порядке не более одного раза

Reduced Ordered BDD - это редуцированная OBDD. Именно ROBDD обладает всеми хорошими свойствами

Чаще всего ROBDD называют просто BDD

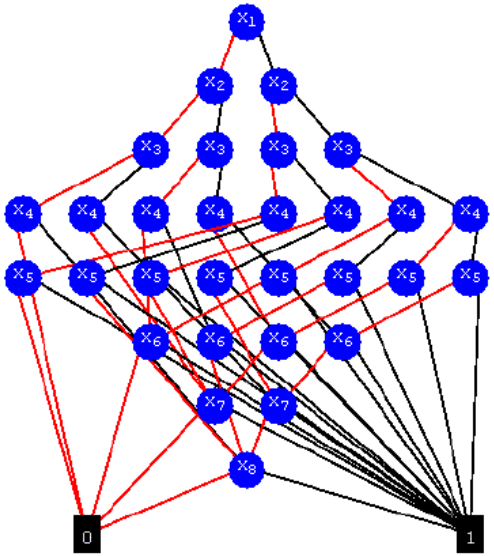
Порядок: x < y < z по одной ветви
x < z < y по другой

R.E.Bryant. Symbolic function manipulation with ordered binary-decision diagrams. *ACM Computing Surveys, 24(3), 1992 (CMU)*

Примеры ROBDD (Reduced Ordered BDD)

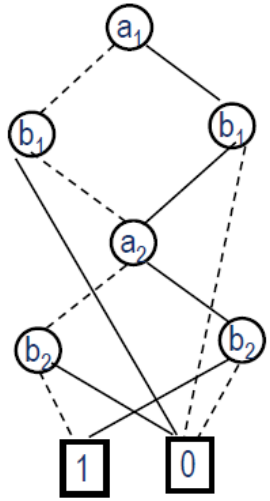
Функция от 8 переменных. ТИ – $2^8 = 256$ строк

$f = x_1x_3x_5 \to x_7 \vee x_2 \to x_4x_5 \vee x_2 \to x_3 \to x_6x_7 \vee x_5x_6x_7 \to x_8$



32 вершины

$f = (a_1 \equiv b_1)(a_2 \equiv b_2)$

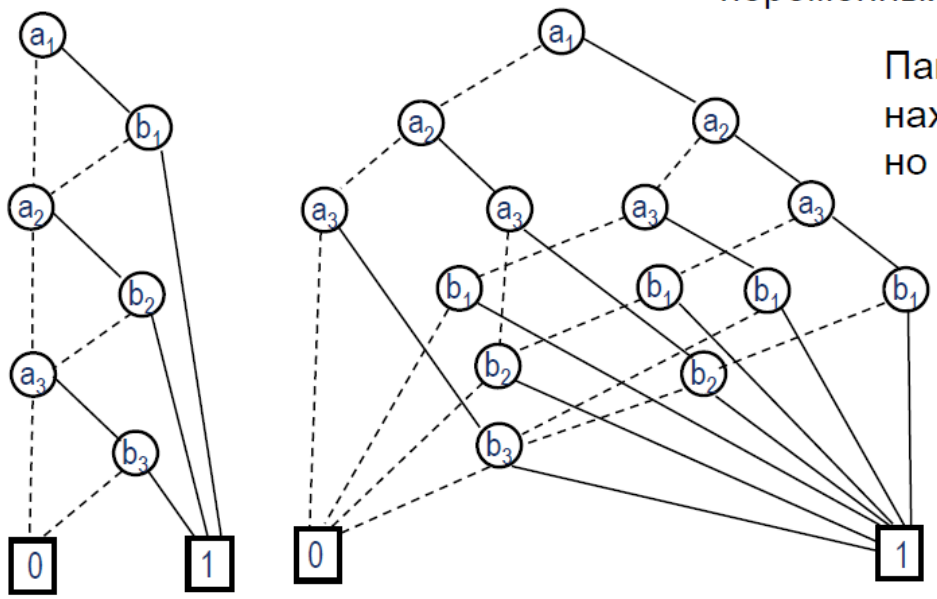


a1<b1<a2<b2

Это – максимальный граф для функций от 8 переменных? ПОЧЕМУ?

Сложность BDD зависит от порядка переменных

$f = a_1b_1 \vee a_2b_2 \vee a_3b_3$



$a_1 < b_1 < a_2 < b_2 < a_3 < b_3$

Рост линеен

$a_1 < a_2 < a_3 < b_1 < b_2 < b_3$

Рост экспоненциален

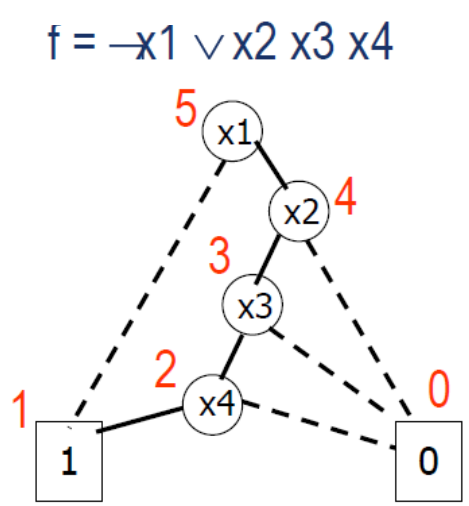
Задача поиска оптимального порядка переменных в BDD является NP трудной

Пакеты работы с BDD легко находят хотя и не оптимальную, но “хорошую” упорядоченность

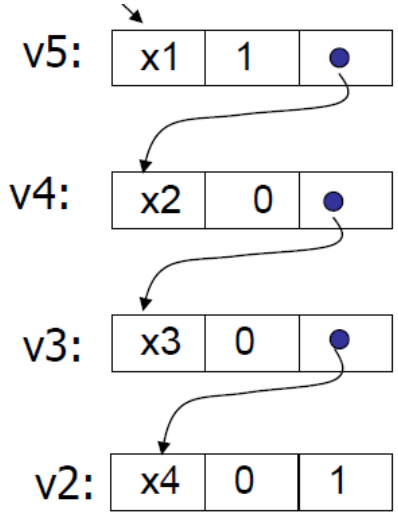
Для почти всех встречающихся на практике булевых функций сложность BDD линейна, либо существуют простые эвристики для установления “хорошего” упорядочения. Существуют алгоритмы динамического переупорядочивания

R.Rundell. Dynamic variable ordering for ordered binary decision diagrams. // Int. Conf. on Computer-Aided Design, IEEE, 1993

Как представить BDD?



n	var	low	high
0			
1			
2	x4	0	1
3	x3	0	2
4	x2	0	3
5	x1	1	4

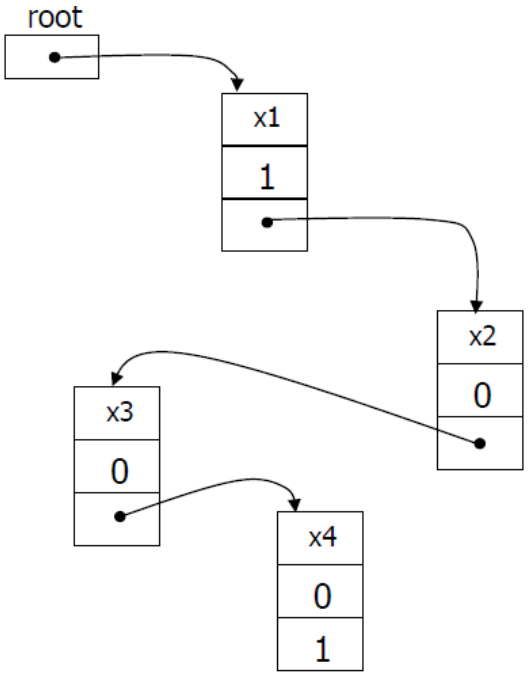


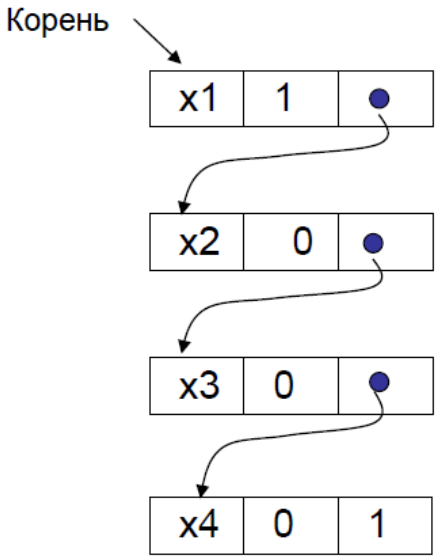
- Таблицей, в которой указываются номер вершины, номер переменной, и куда направлены два выхода, при значении переменной 0 и при ее значении 1
- Связным списком
- Множеством формул для вершин {v: [x →p0, p1]} , где x – переменная, помечающая вершину v, а p0 и p1 – указатели на подграфы, в которые из вершины v идут ребра, помеченные, соответственно, 0 и 1.

f = {v5: [x1 →1,v4], v4: [x2 →0,v3], v3: [x3 →0,1], v2: [x4 →0,v2] }

Сложность представления БФ в BDD пропорциональна числу вершин

Представление BDD

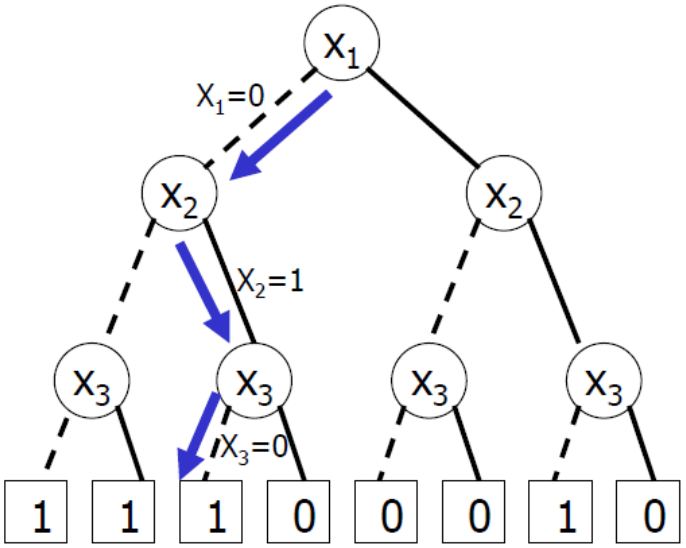


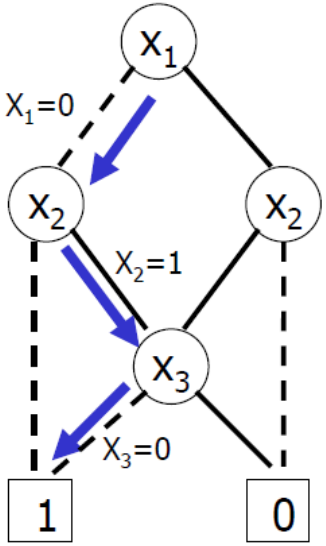


Как вычислять значение двоичной функции по бинарному решающему дереву (решающему графу)

Вычисление f на наборе <0,1,0>:
f(0,1,0) = 1

$x_1x_2x_3$	$f(x_1,x_2,x_3)$
000	1
001	1
010	1
011	0
100	0
101	0
110	1
111	0





ТИ

Binary Decision Tree

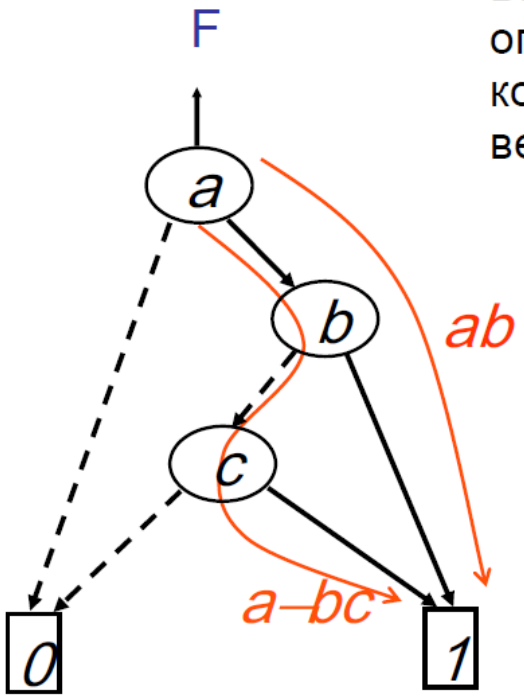
Binary Decision Diagram

Значение функции вычисляется простым движением по ветвям от корня к листу – по дереву (или диаграмме) решений

BDD → ДНФ

$F(a,b,c) = ab \vee a\bar{b}c$

a	b	c	f(a,b,c)	
000			0	
001			0	
010			0	
011			0	
100			0	
101			1	f2 = a–bc
110			1	f1 = ab
111			1	

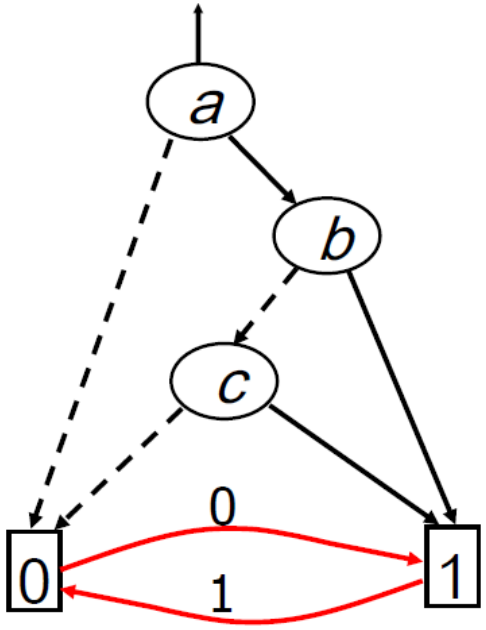


Вид функции в ДНФ определяют те пути, которые из корневой вершины идут в 1

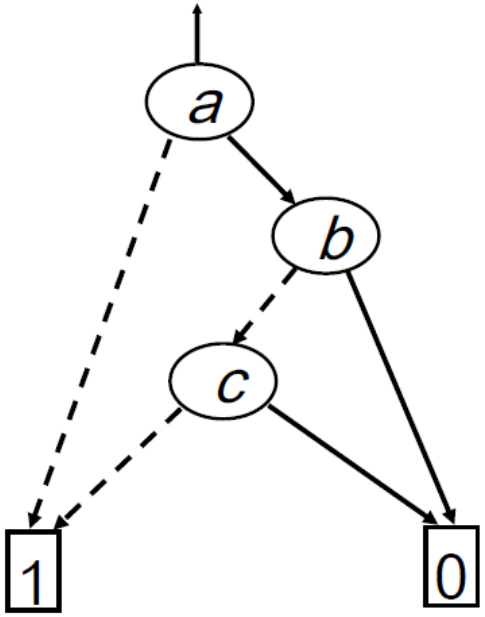
$F = ab \vee a\bar{b}c$

Булевы операции над BDD: отрицание BDD

F1=ab ∨ a–bc

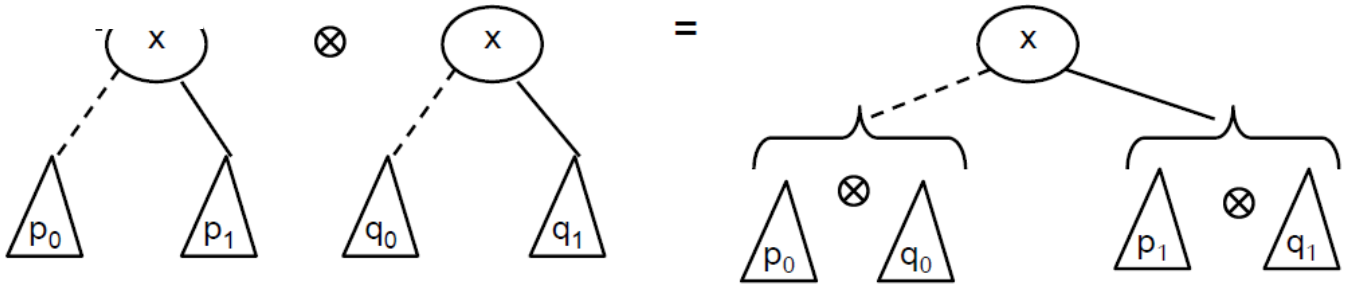


F2= ¬(ab ∨ a–bc)

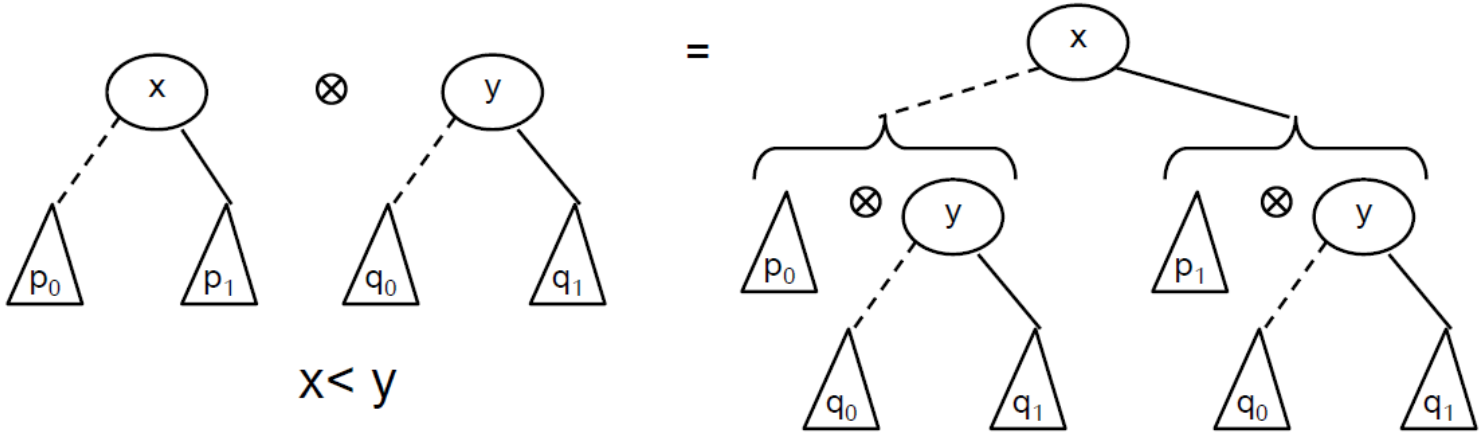


- BDD для отрицания функции строится изменением значения 0 на 1 и 1 на 0 в финальных вершинах

Булевы операции над BDD: алгоритм Apply



$$[x \to p_0, p_1] \otimes [x \to q_0, q_1] = [x \to p_0 \otimes q_0 , p_1 \otimes q_1]$$

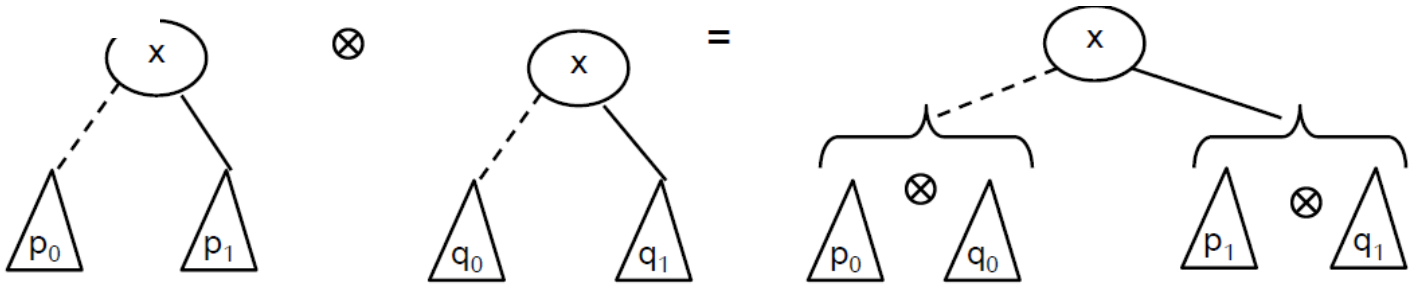


x< y

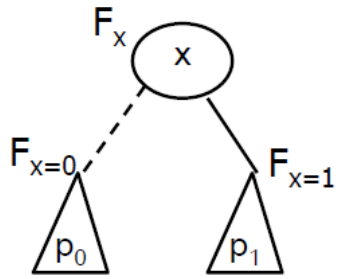
$$[x \to p_0, p_1] \otimes [y \to q_0, q_1] = [x \to p_0 \otimes [y \to q_0, q_1], p_1 \otimes [y \to q_0, q_1]\ ]$$

Реализация булевых операции над BDD имеет линейную сложность

Доказательство корректности алгоритма Apply



BDD основано на разложении Шеннона:
$F(\ldots, x, \ldots) = \neg x\, F_{x=0} \vee x\, F_{x=1}$



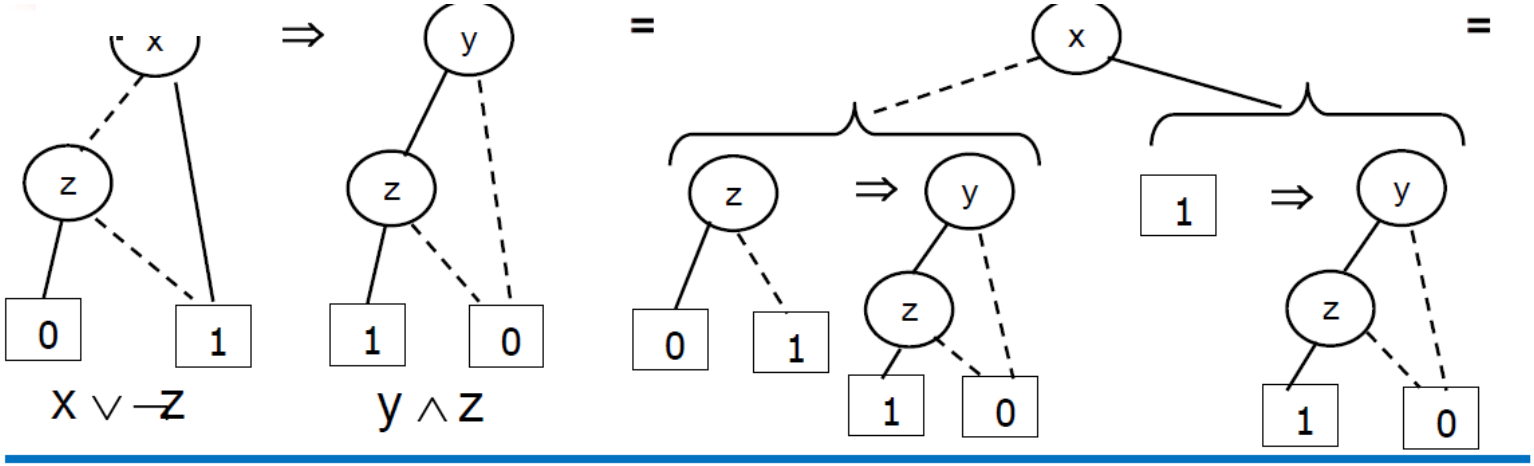
Доказательство корректности алгоритма Apply сводится к доказательству справедливости равенства:

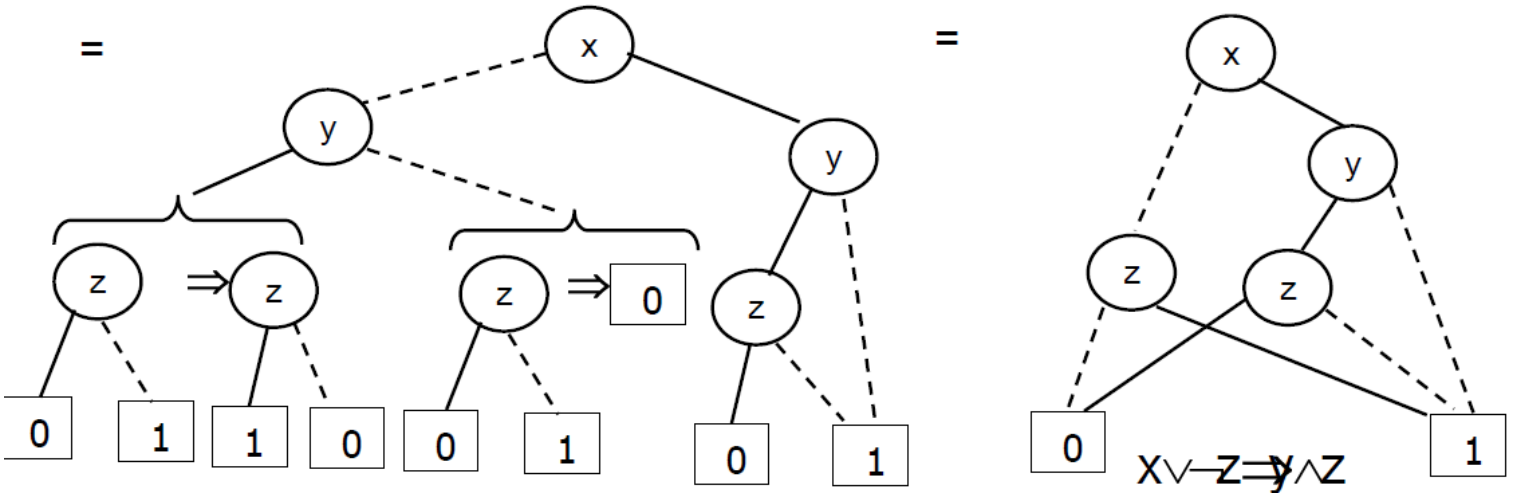
$$(\neg x\, F_{x=0} \vee x\, F_{x=1}) \otimes (\neg x\, \Phi_{x=0} \vee x\, \Phi_{x=1}) =(?)\ \neg x\, (F_{x=0} \otimes \Phi_{x=0}) \vee x\, (F_{x=1} \otimes \Phi_{x=1})$$

Задача: проверить справедливость этого соотношения для операции $\otimes$ соответствующей операциям $\{\vee, \wedge, \Rightarrow \Leftrightarrow \ldots\}$

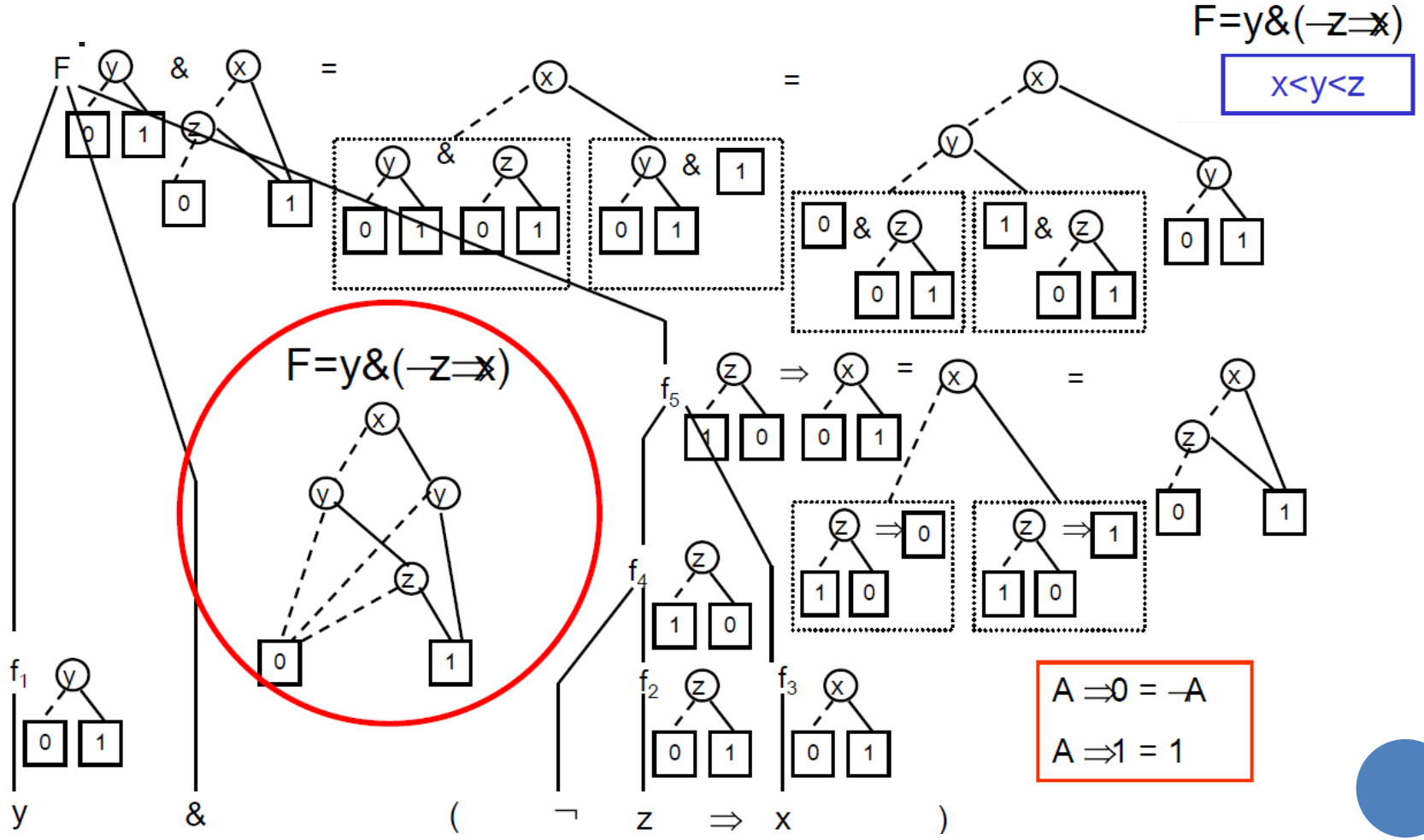
Пример: BDD(xv¬z) →BDD(y &z) = BDD ((xv ¬ z) →(y & z))







Как построить BDD по булевой функции



Свойства BDD

1. BDD – каноническое представление – *любая логическая функция имеет единственное представление, минимальное в базисе {0, 1, ite} при данном порядке переменных* (этим не обладают КНФ и ДНФ!!!)

2. Сложность зависит от порядка переменных. Разный порядок переменных дает разные представления

Пример: функция $(a_1 \oplus b_1) \& \ldots \& (a_n \oplus b_n)$

при порядке $a1 \ldots a_n\ b_1 \ldots b_n$ имеет сложность $3 \times 2^n - 1$ - экспоненциальная

при порядке $a_1 b_1 a_2 b_2 \ldots a_n b_n$ имеет сложность $3 \times n + 2$ - линейная

3. Проблема нахождения оптимального порядка NP – трудна. Но есть эвристики

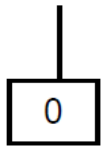
4. BDD для многих практически важных функций имеет линейную сложность. *Некоторые классы* функций имеют экспоненциальную сложность BDD *при любом* порядке переменных (пример: функция, выдающая средний бит результата произведения A × B n-разрядных переменных A и B)

5. Булевы операции над БФ, представленными в BDD, просты.
Теорема. *Сложность выполнения булевых операций над двумя функциями f и g, представленными в BDD, полиномиальна: O(|f| x |g|)*

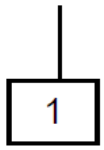
Свойства BDD

- Выполнимость (Теорема Кука: "*Проблема выполнимости булевой формулы NP-полна*")
  - Для BDD проверка выполнимости, невыполнимости и общезначимости тривиальны – не является ли BDD вырожденной (значения 0 или 1)
  - Но проблема построения представления функции в BDD является NP-полной

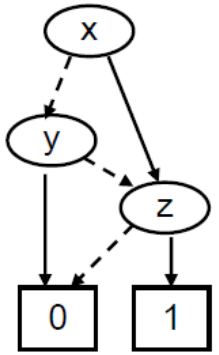
Вырожденные BDD:



Невыполнима



Общезначима



Выполнима

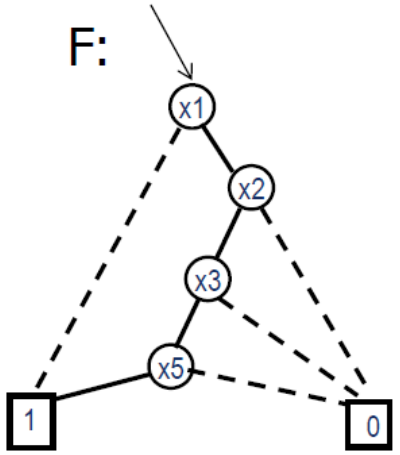
Существенные переменные. Проблема зависимости двоичной функции от конкретной переменной

- Проблема зависимости двоичной функции от конкретной переменной – классическая проблема теории двоичных функций

$F = \bar{x}1\bar{x}2\bar{x}3\,x4\bar{x}5 \vee \bar{x}1\bar{x}2\bar{x}3\bar{x}4\bar{x}5 \vee x1x2x3\bar{x}4x5 \vee x1x2x3x4x5$

Все переменные присутствуют в ДНФ функции F. Но какие переменные являются "существенными" (от каких переменных функция действительно зависит)?

Как узнать?
По СДНФ?
По ДНФ?
По ТИ?



F не зависит от y4

Булева функция зависит от переменной x тогда и только тогда, когда в ее BDD присутствует вершина, помеченная x

Библиотеки

ABCD: The ABCD package by Armin Biere
http://fmv.jku.at/abcd/
BuDDy: A BDD Package by Jørn Lind-Nielsen.
http://sourceforge.net/projects/buddy/
CMU BDD, BDD package, Carnegie Mellon University, Pittsburgh
http://www.cs.cmu.edu/~modelcheck/bdd.html
CUDD: BDD package, University of Colorado, Boulder
http://vlsi.colorado.edu/~fabio/CUDD/
JavaBDD, a Java port of BuDDy that also interfaces to CUDD, CAL, and JDD
http://javabdd.sourceforge.net/
The Berkeley CAL package which does breadth-first manipulation
http://embedded.eecs.berkeley.edu/Research/cal_bdd/
BDDFunctions: Библиотека многокорневых BDD (СПб университет)
http://www.sysprog.info/2010/09.pdf

Программа визуализации BDD:

http://www.cs.uc.edu/~weaversa/BDD_Visualizer.html

Binary Decision Diagrams (nicze.de) https://nicze.de/philipp/bdds/

Применения BDD

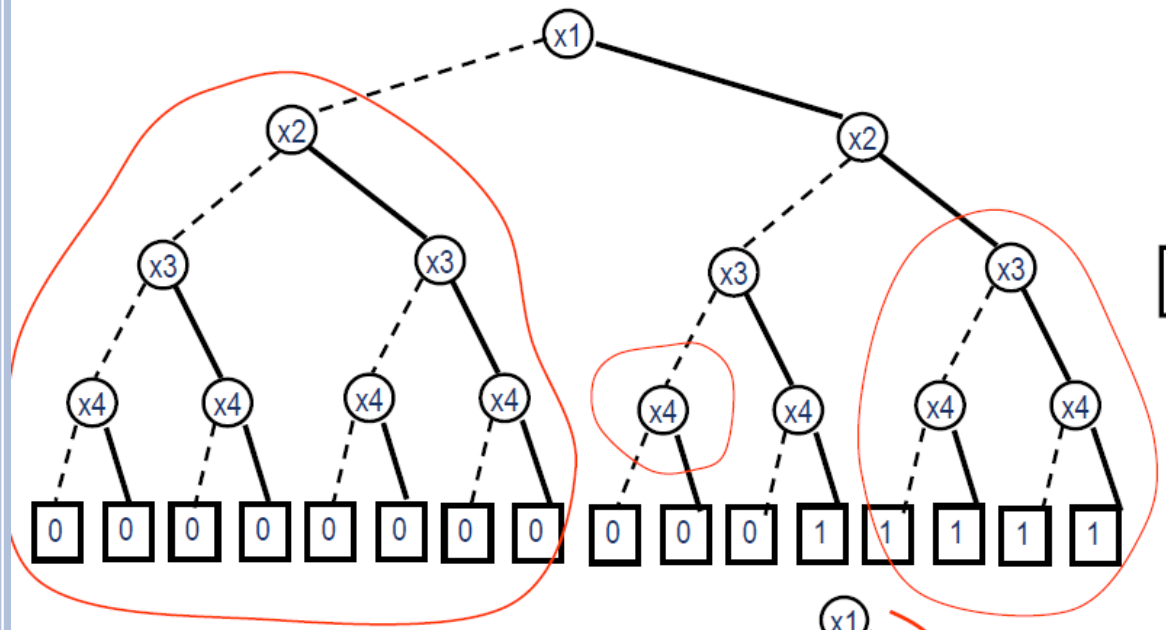
- Применений оказалось огромное количество –
  везде, где, в принципе, возможно применение двоичных функций,
  использование BDD как формы представления двоичных функций
  качественно повышает эффективность решений

- "Прямые" применения
  - Минимизация двоичных функций
  - Операции над БФ от большого числа переменных
  - Синтез логических схем по их представлению в форме BDD
  - Верификация и проверка эквивалентности логических схем
- Борьба с "проклятием размерности"
  - Упрощение любых алгоритмов, манипулирующих конечными структурами данных большого объема
    - Компрессия изображений, Поиск в БД, Алгоритмы на графах
      ...
  - Верификация реагирующих (reactive) систем
    - С использованием BDD стало возможным увеличить сложность верифицируемых систем на многие порядки!

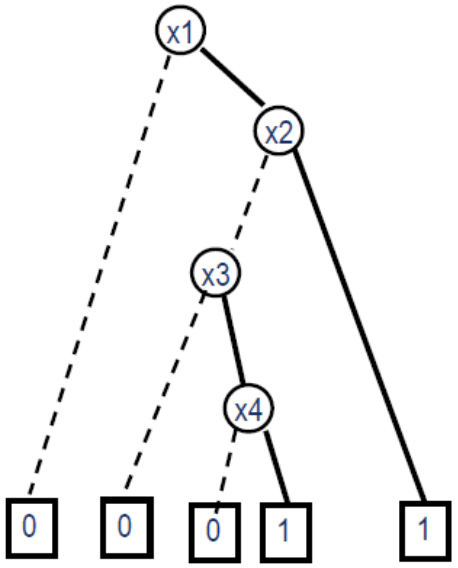


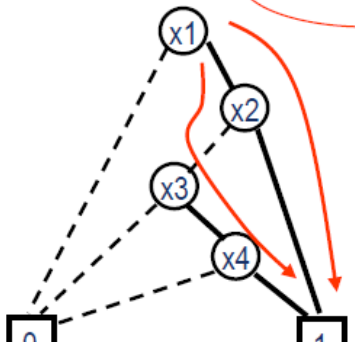
Минимизация двоичных функций

$F = x_1 \sim x_2 x_3 x_4 \vee x_1 x_2 \sim x_3 x_4 \vee x_1 x_2 \sim x_3 \sim x_4 \vee x_1 x_2 x_3 \sim x_4 \vee \sim(\sim x_1 x_2 \sim x_3 x_4)$



$F = x1(x2 \vee x3x4)$

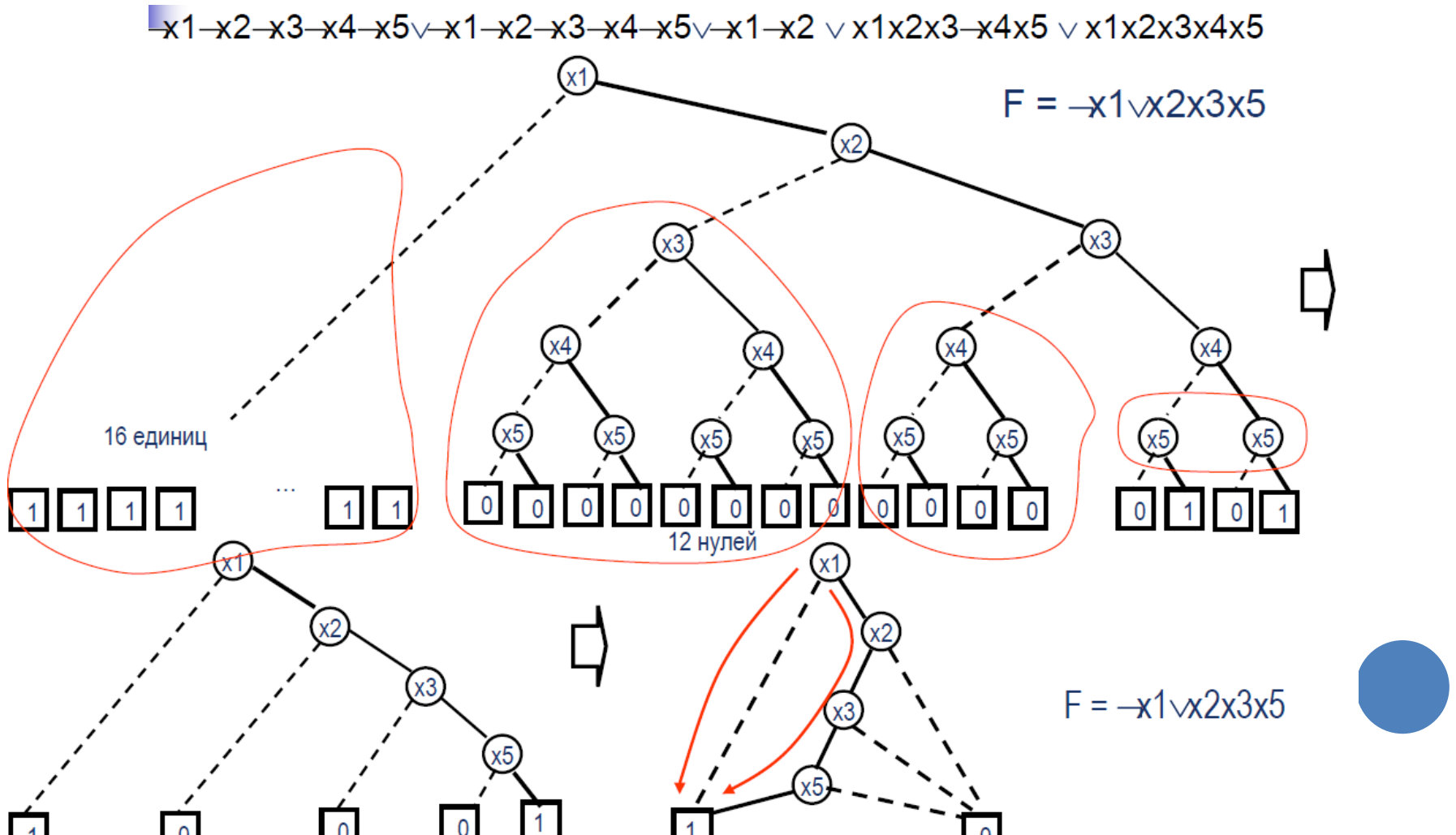




$F = x1(x2 \vee \sim x2x3x4)$

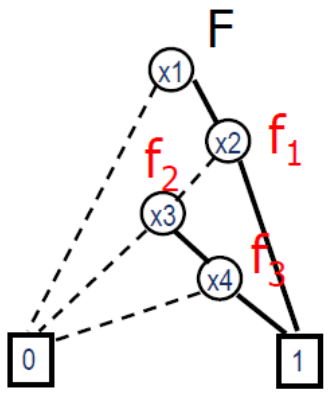
$= x1(x2 \vee x3x4)$

Минимизация двоичных функций



Минимизация двоичных функций

$F = \bar{x}_1 \sim x_2 x_3 x_4 \vee x_1 x_2 \sim x_3 x_4 \vee x_1 x_2 \sim x_3 \sim x_4 \vee x_1 x_2 x_3 \sim x_4 \vee \sim(\sim x_1 x_2 \sim x_3 x_4)$

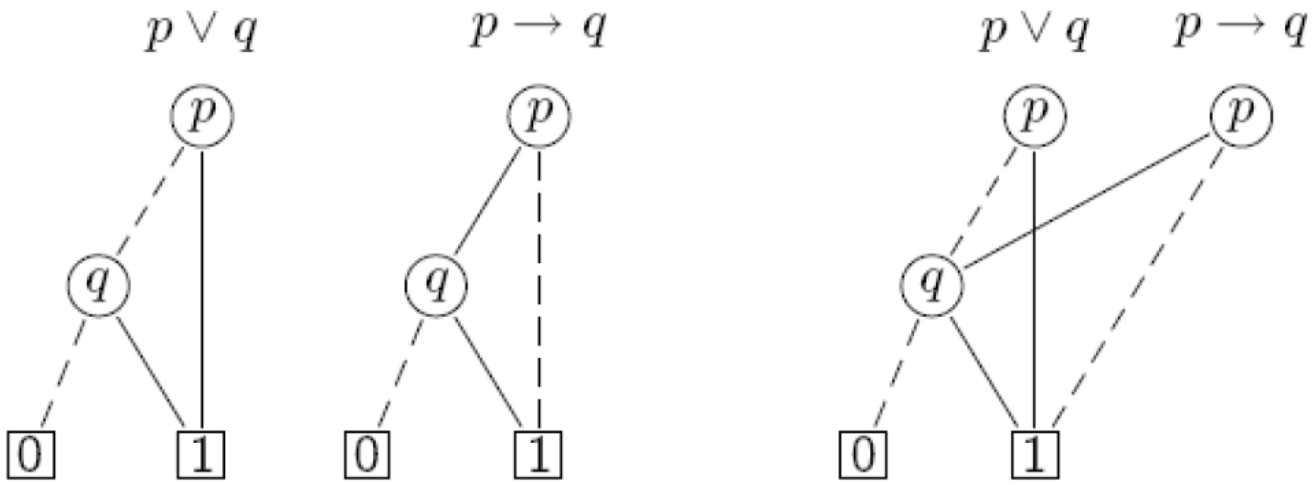


F = if x1 then f_1 else 0
f_1 = if x2 then 1 else f_2
f_2 = if x3 then f_3 else 0
f_3 = if x4 then 1 else 0

F = if x1 then f_1 else 0
F = if x1 then (if x2 then 1 else f_2) else 0
F = if x1 then (if x2 then 1 else (if x3 then f_3 else 0)) else 0
F = if x1 then (if x2 then 1 else (if x3 then (if x4 then 1 else 0) else 0))else 0

BDD – это минимальное представление БФ в классе функций {ite, 0, 1}

Совместная минимизация нескольких БФ

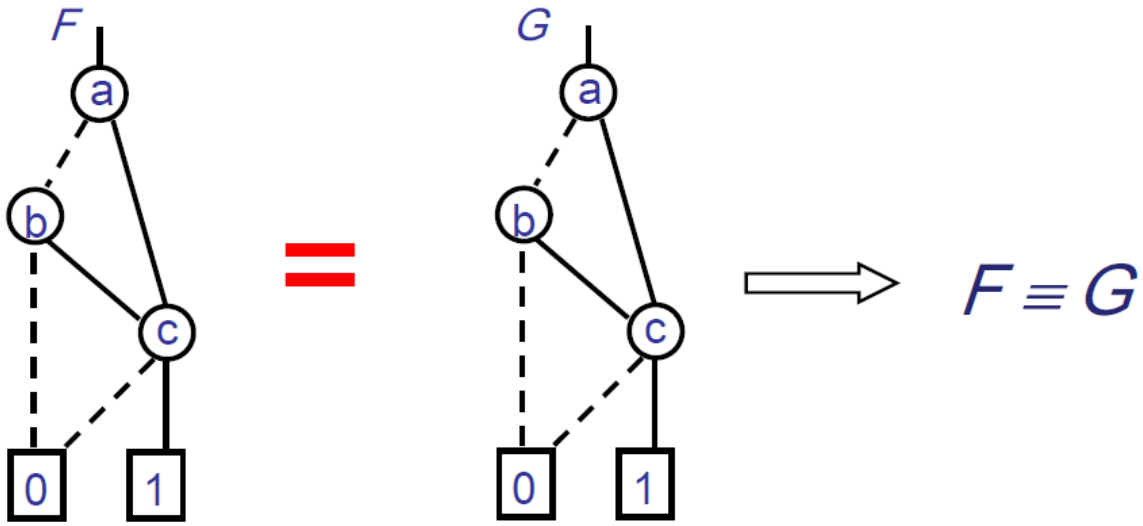


- Многокорневые BDD - представление нескольких функций с помощью BDD с несколькими корневыми вершинами – это, фактически, совместная минимизация нескольких функций в классе функций {if-then-else, 0, 1}

Недавняя разработка в Петербургском Университете – библиотека многокорневых BDD http://www.sysprog.info/2010/09.pdf

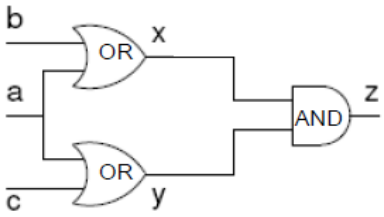
Проверка равнозначности булевых функций

$F = a\bar{b}c \vee abc \vee ab\bar{c}$ $\quad G = ac \vee bc$ $\quad F \equiv G\ (?)$



Поскольку BDD – каноническое представление, проверка равнозначности двух функций, представленных в BDD, сводится к проверке совпадения направленных графов

Как построить BDD по структуре схемы

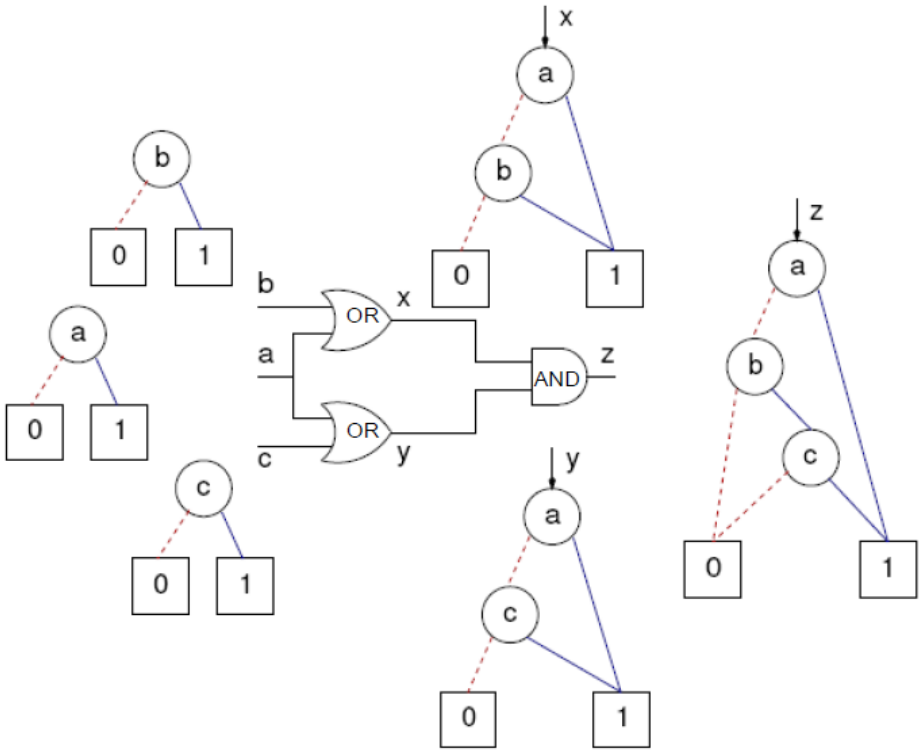


1. Сначала строятся BDD исходных переменных

2. Затем по ярусам строятся BDD всех промежуточных подфункций

3. Результирующая BDD представляет БФ, реализуемую схемой

Как построить BDD по структуре схемы

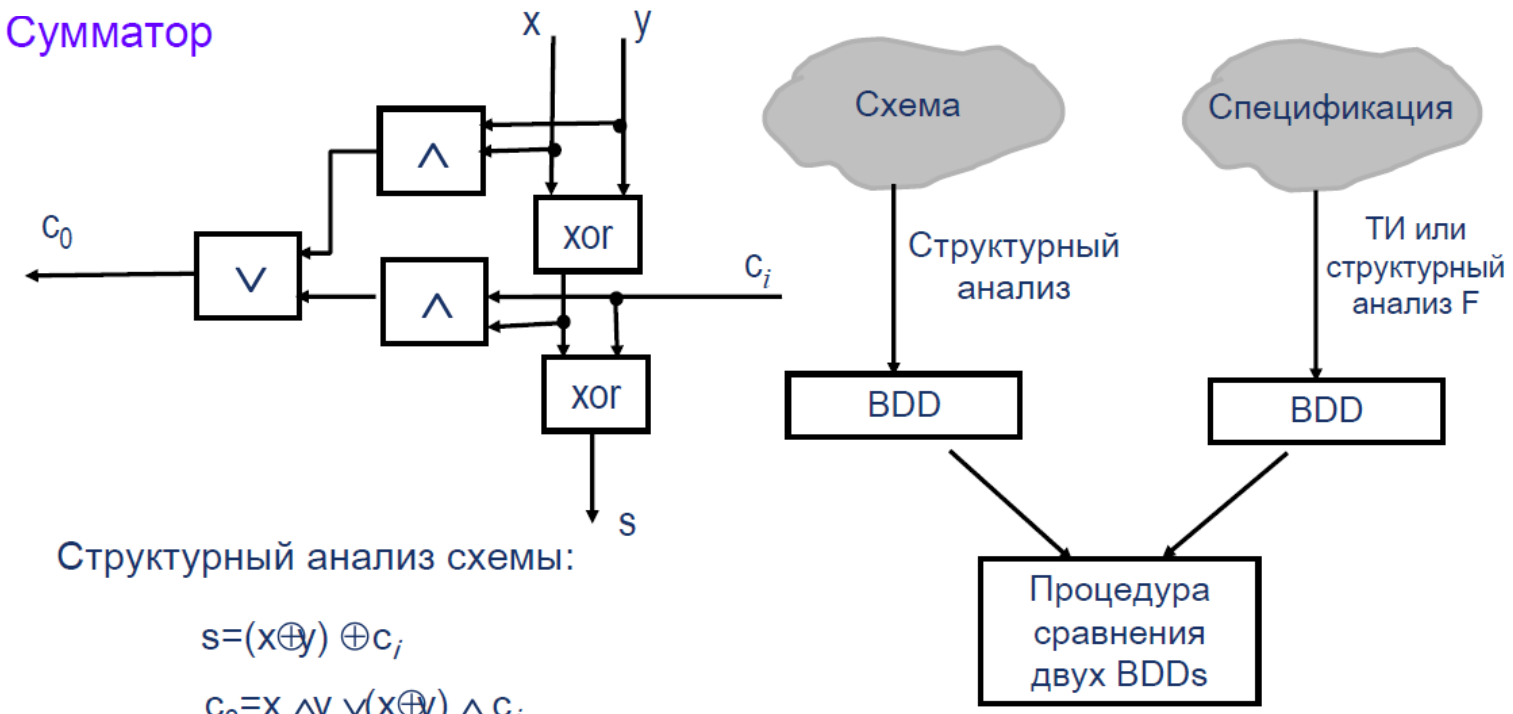


1. Сначала строятся BDD исходных переменных

2. Затем по ярусам строятся BDD всех промежуточных подфункций

3. Результирующая BDD представляет ЛФ, реализуемую схемой

Применение BDD: удовлетворяет ли логическая схема ее спецификации?



Структурный анализ схемы:

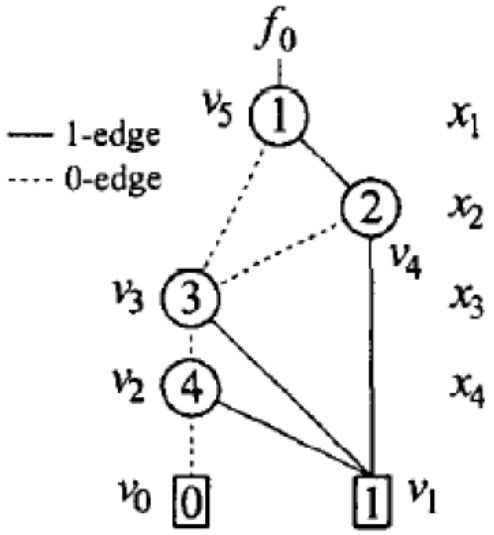
$s=(x \oplus y) \oplus c_i$

$c_0=x \wedge y \vee (x \oplus y) \wedge c_i$

Спецификация: ТИ или формула

Аналогично проверяется эквивалентность двух схем

Автоматическая генерация программы для вычисления значений функции по BDD



(a) BDD.

v_5: if(x_1 == 0) goto v_3;
else goto v_4;
v_4: if(x_2 == 0) goto v_3;
else goto v_1;
v_3: if(x_3 == 0) goto v_2;
else goto v_1;
v_2: if(x_4 == 0) goto v_0;
else goto v_1;
v_1: return(1);
v_0: return(0);

(b) Branching program generated from BDD.

Программа вычисляет значение БФ при заданных аргументах по BDD, имитируя движение по упорядоченному графу.

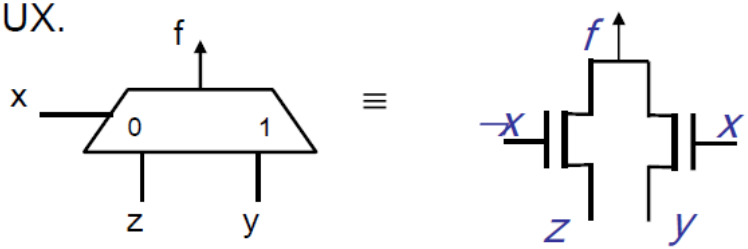
Ответ всегда найдется не более, чем за N шагов

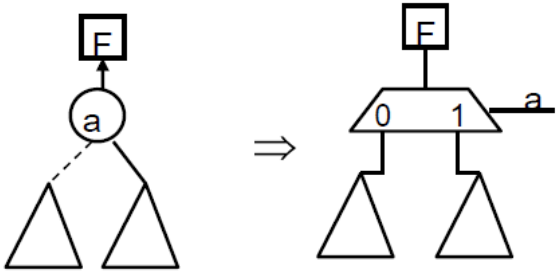
Типовая задача нашего следующего курса: Построение компиляторов

Реализация схемы по BDD

Используется мультиплексор (MUX) – схема, с тремя входами и одним выходом

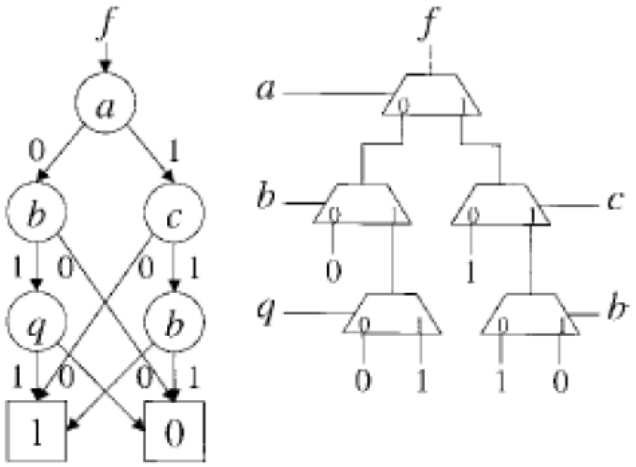
Существует технология PTL (Pass Transistor Logic) построения ЛС на основе MUX.





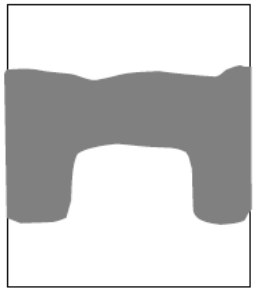
Произвольная BDD может быть реализована схемно непосредственно по структуре графа заменой каждого его узла на мультиплексор

PTL синтез, основанный на "прямом" BDD mapping

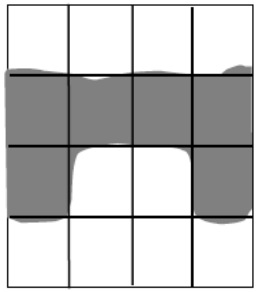


"The construction of circuits resulting from a direct mapping of BDDs has gained great interest, since the graph directly corresponds to a multiplexor network" (Из руководства по синтезу схем 2011)

Компрессия изображений



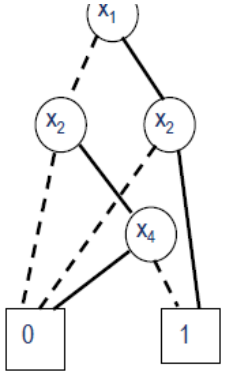
а)



б)

x_1x_2 \ x_3x_4	10	11	01	00
10	0	0	0	0
11	**1**	**1**	**1**	**1**
01	**1**	0	0	**1**
00	0	0	0	0

в)



г)

$f = x1x2 \vee x2 \rightarrow x4$

а) двумерное черно-белое изображение

б) разбиение изображения на множество пикселей

в) соответствующая двоичная функция $x_1x_2 \vee x_2 \rightarrow x_4$

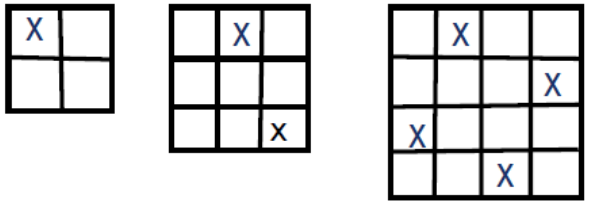
г) BDD, представляющая изображение

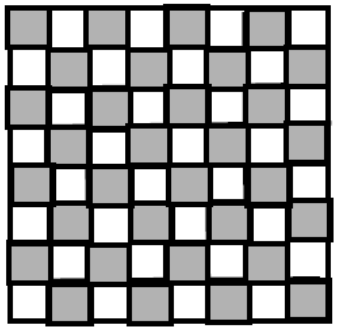
Для представления мегабайтного изображения нужна BDD функции от 23 переменных (поскольку $\lceil \log (8*10^6) \rceil = 23$)

Во многих случаях такое представление экономнее стандартных, алгоритмов сжатия, например JPEG

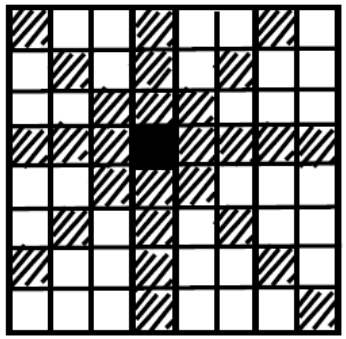
Логические задачи, решаемые с помощью БФ

Задача о расстановке ферзей





?



Если ферзь в клетке (i,j), *ТО:*
$(x_{ij} \Rightarrow$:

на *i*-й горизонтали нет ферзей: $\bigwedge_{1\le k\le N,\ k\ne j} \neg x_{ik}$

И на j-й вертикали нет ферзей: $\bigwedge_{1\le k\le N,\ k\ne i} \neg x_{kj}$

И на с-з диагонали нет ферзей: $\bigwedge_{1\le k\le N,\ 1\le j+k-j\le N,\ k\ne i} \neg x_{k,j+k-i}$

И на с-в диагонали нет ферзей: $\bigwedge_{1\le k\le N,\ 1\le j+I-k\le N,\ k\ne i} \neg x_{k,j+i-k}$

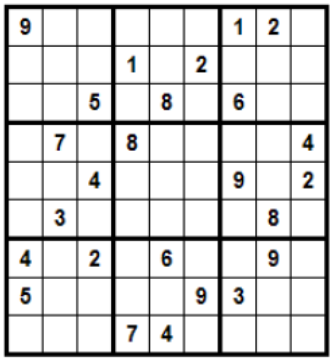
И на каждой горизонтали есть ферзь: $\bigwedge_{1\le i\le N} x_{i1} \vee x_{i2} \vee x_{i3} \ldots \vee x_{iN}$

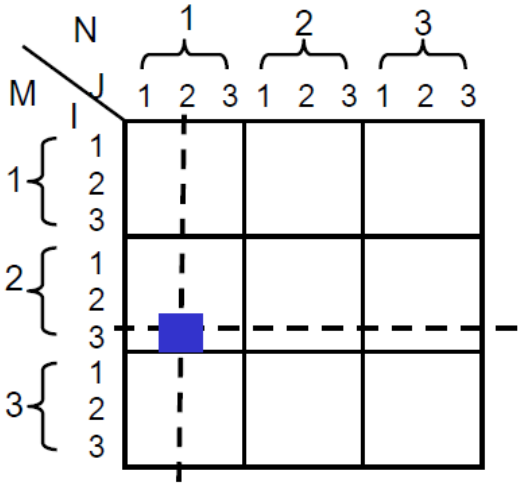
Конъюнкция всех этих формул даст такую БФ F, что любая интерпретация, на которой F=1, будет решением

Для решения задачи нужна BDD для ф-ции от 64 переменных

Пример: сложные комбинаторные задачи

Судоку:





Цифры от 1 до 9, все разные:

на каждой горизонтали,
на каждой вертикали
и в каждом квадрате

$x_{MNIJK}=1$, if $a_{MNIJ}=K$

729 переменных

Пример:

$a_{2312} = 3$ кодируется, как $x_{23123}=1$

$M, N, I, J, m, n, i, j \in \{ 1, 2, 3 \}$

В каждой клетке – по крайней мере одна цифра: $A= \forall M \forall N \forall I \forall J \exists K\ x_{MNIJK}$

Горизонтали: $Г= \forall M \forall N \forall I \forall J \forall K \forall m \forall i\ [\ m \neq M \vee i \neq I \Rightarrow (x_{MNIJK} \Rightarrow x_{m N i J K})\ ]$

Вертикали: $В= \forall M \forall N \forall I \forall J \forall K \forall n \forall j\ [\ n \neq N \vee j \neq J \Rightarrow (x_{MNIJK} \Rightarrow x_{M n I j K})\ ]$

Квадраты: $Д= \forall M \forall N \forall I \forall J \forall K \forall i \forall j\ [\ i \neq I \vee j \neq J \Rightarrow (x_{MNIJK} \Rightarrow x_{M N i j K})\ ]$

Любой набор переменных, на котором $A \wedge Г \wedge В \wedge Д = 1$, дает решение задачи

Задачи, которые можно решать с помощью BDD

3	*	7	4	*	*	2	*	*
*	*	4	*	*	*	*	*	*
*	*	*	*	*	*	5	1	*
*	*	*	*	1	3	*	9	*
*	*	*	5	9	*	*	*	8
*	*	*	2	*	*	*	*	*
7	6	*	*	2	*	*	*	*
*	*	*	8	*	*	*	2	*
9	1	*	*	3	*	*	*	*

- Эта задача опубликована как "Судоку для гениев". Утверждается, что такие сложные судоку может решить только 3% населения нашей планеты. Если вы ее решите, можете считать себя гением!

		5	3					
8							2	
	7			1		5		
4					5	3		
	1			7				6
		3	2				8	
	6		5					9
		4					3	
					9	7		

- Эта задача создана математиком Арто Инкала, который утверждает, что она самая сложная в мире

В 2012 г. Проф. Gary McGuire из University Dublin предложил доказательство того, что минимальное число стартовых цифр Судоку, чтобы решение было единственным, составляет 17.
McGuire и его команда потратили 7 млн часов времени CPU в Ирландском Центре Высокопроизводительных компьютеров в Дублине, чтобы это доказать

Программирование в ограничениях

Constrained programming (Constraint Satisfaction Problem, CSP)

Основано на описании модели задачи, а не алгоритма ее решения

Модель специфицируется в виде отношений – ограничений, которые отражают связи, существующие между параметрами задачи

Постановка задачи

На переменные $Z_1,..., Z_n$ ($Z_i \in D_i$) наложены ограничения $C_k(Z_1, ..., Z_n)$ - уравнения, неравенства, логические выражения и т.п

Найти наборы значений $<a_1, ..., a_n>$ ($a_i \in D_i$), удовлетворяющие ограничениям

Использование BDD:

Значения в каждой области D_i кодируются двоичными наборами

Каждое ограничение C_k преобразуется в логическую функцию f_k , которая представляется в форме BDD

Множество ограничений представляется конъюнкцией этих функций $F = \&_i f_i$

Решение определяется такими наборами кодировок, на которых F =1



Задача Эйнштейна

Эйнштеин считал, что 98% жителей Земли не в состоянии ее решить В УМЕ.
Принадлежите ли вы к 2% самых умных людей планеты?



Вопрос:

кто держит рыбок?

В 5 домах разного цвета живет по одному человеку разных национальностей. Каждый жилец пьет только один определенный напиток, курит определенную марку сигарет и держит определенное животное.

Никто из 5 человек не пьет одинаковые с другими напитки, не курит одинаковые сигареты и не держит одинаковое животное.

Известно, что:

Англичанин живет в красном доме
Швед держит собаку
Датчанин пьет чай
Зеленый дом стоит слева от белого
Жилец зеленого дома пьет кофе
Человек, который курит Pall Mall, держит птицу
Жилец из среднего дома пьет молоко
... ...

Программирование в ограничениях (Constrained Programming, Constraint Satisfaction Problem— CSP)

- **Программирование в ограничениях** является парадигмой программирования, в которой отношения между переменными указаны в форме ограничений. Программирование в ограничениях является формой декларативного программирования.
  В отличие от императивного программирования, решение определяется не последовательностью шагов для исполнения, а свойствами искомого решения, определяемыми набором ограничений.

В нашей постановке:

N объектов, M параметров,

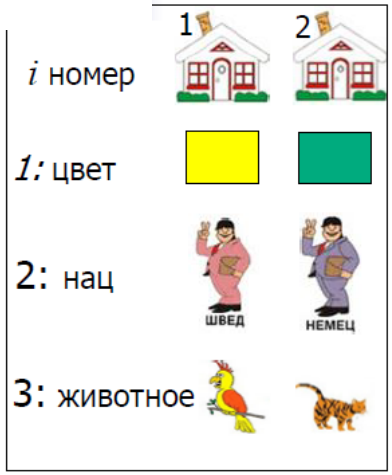
- каждый параметр имеет N значений;
- задаются некоторые факты - ограничения на соотношения между значениями параметров

Решение – наборы значений параметров объектов, удовлетворяющие ограничениям

Решений может быть 0, 1 и множество



БФ для представления условий задач CSP (один из возможных подходов)



- Ограничения:

1. Каждый объект имеет какое-то значение параметров
2. Объекты имеют разные значения параметров
3. Швед любит птичку
4. 2-й дом не красный

… … …

x_{ijk} : у дома i параметр j равен k

$x_{121} = 1$: в доме 1 живет швед;
$x_{211} = 0$: дом 2 НЕ желтый

N объектов, M параметров, каждый по N значений
N×M×N двоичных переменных. У нас 12 переменных

$F(x_{111}, x_{112}, x_{121}, x_{122}, x_{131}, x_{132}, x_{211}, x_{212}, x_{221}, x_{222}, x_{231}, x_{232})$

$F(0, 1, 1, 1, 0, 1, 1, 0, 0, 1, 0, 0) = 1$ означает:

Дом 1 – не желтый, он красный, в нем живут и швед и немец, в нем не любят птичку, а любят кота.
Дом 2 – желтый, он не красный, в нем не живет швед, живет немец, в нем не любят птичку и не любят кота.

Ограничения нужно выразить функциями

Задача Эйнштейна с использованием БФ

D^1: Цвет = {*красный, синий, желтый, зеленый, белый*}
D^2: Нац = {*англ, немец, японец, норвежец, швед*}
D^3: Питье = {*сок, вино, чай, молоко, вода*}
D^4: Животное ={*улитка, лиса, кошка, лошадь, собака*}
D^5: Сигареты = {"Парламент", "Кулс", …, "Честерфилд"}

$Q_1, \ldots, Q_5$ – объекты (дома)

X_{ijk} – i-й объект
j-й параметр
равен k

(125 двоичных переменных)

Ограничения:

F_1: $\bigwedge_{i=1..5} \bigwedge_{j=1..5} \bigvee_{k=1..5} x_{ijk}$ – каждый объект имеет какие-то значения параметров

F_2: $\bigwedge_{i=1..5} \bigwedge_{j=1..5} \bigwedge_{k=1..5} \bigwedge_{s=1..5} [(i \neq s) \Rightarrow (x_{ijk} \Rightarrow \neg x_{sjk})]$ - объекты имеют разные значения параметров

Известные факты:

f_1: $\bigwedge_{i=1..5} (x_{i21} \Leftrightarrow x_{i11})$ – англ $\Leftrightarrow$ в красном

f_2: $\bigwedge_{i=1..5} (x_{i25} \Leftrightarrow x_{i45})$ - швед $\Leftrightarrow$ держит собаку

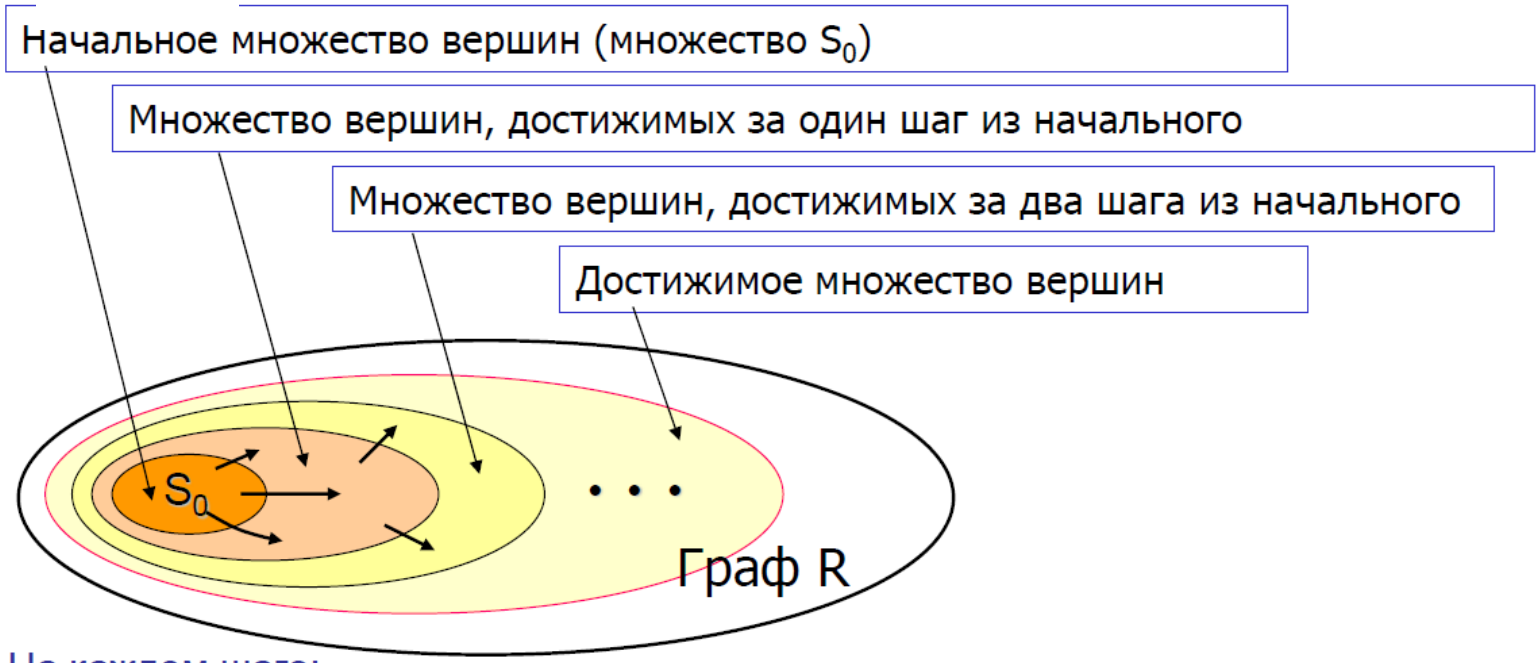
… …

Конъюнкция всех ограничений дает функцию, единичное значение которой - на допустимых вариантах

Возможные решения - наборы значений параметров, на которых
$\Phi = F_1 F_2 f_1 f_2 f_3 \ldots = 1$

BDD для БФ от 125 переменных решает задачу

Определение множества вершин, достижимых из заданного подмножества вершин S_0 на заданном направленном графе



На каждом шаге:

а) Для текущего множества Si определяем множество Ti вторых элементов ограничения R на множестве S_i

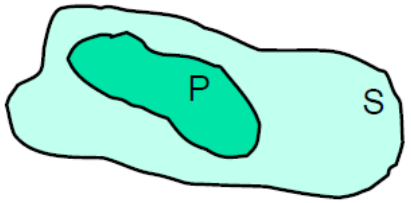
б) Строим множество S_{i+1} =Si ∪Ti

Повторяем операции до тех пор, пока S_i и S_{i+1} не совпадут

Конечные структуры данных: характеристическая функция подмножеств конечного множества

$x_1x_2x_3$	f
000	1
001	1
010	1
011	0
100	0
101	0
110	1
111	0

Пусть S – конечное множество из m элементов, $k=\lceil \log_2(m) \rceil$. Закодируем все элементы S наборами двоичных переменных $x_1, x_2, \ldots, x_k$.



Пример: S - из 8 элементов: $\{a_0, a_1, \ldots, a_7\}$.
Кодируем (k=3): $a_0 \Leftrightarrow 000$, $a_1 \Leftrightarrow 001$, … , $a_7 \Leftrightarrow 111$

Пусть $P \subseteq S$, $P= \{a_0, a_1, a_2, a_6\}$. Соответствующее множество двоичных кодов { 000, 001, 010, 110}.

Булева функция f может задать множество элементов - подмножество конечного множества, если она равна 1 на наборах, кодирующих элементы, которые в это множество входят.
Эта функция называется характеристической функцией множества.
Обозначим χ_P характеристическую функцию множества P.

Любое подмножество конечного множества может быть задано с помощью характеристической БФ. Такое задание называется символьным представлением множества

Характеристические булевы функции конечных множеств можно представить в форме BDD

Обозначим χ_A характеристическую булеву функцию множества A. Она равна 1 на наборах, кодирующих элементы из A, т.е. на { 000, 001, 010, 110}

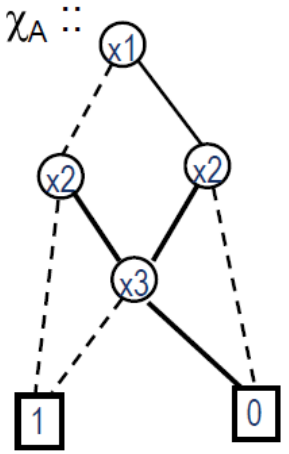
	f
000	1
001	1
010	1
011	0
100	0
101	0
110	1
111	0

Пусть x_1, x_2, x_3 – разряды кодировки.

Тогда:

$\chi_A = \neg x_1 \neg x_2 \neg x_3 \vee \neg x_1 \neg x_2 x_3 \vee \neg x_1 x_2 \neg x_3 \vee x_1 x_2 \neg x_3$

задает A = {000, 001, 010, 110}



Любые подмножества конечного множества можно задавать булевой формулой, представляющей характеристическую функцию, и, следовательно, ее можно представить в форме BDD

Будем писать $\chi_A(V)$, если χ_A представлена , как функция от переменных $V=(x_1, x_2, x_3)$, чтобы показать переменные кодировки и их порядок

Символьные вычисления

- Вычисления с множествами, в которых вместо теоретико-множественных операций выполняются булевы операции над представляющими эти множества характеристическими двоичными функциями, называются символьными вычислениями
- Сведение проблем, связанных с конечными множествами, к решению проблем относительно двоичных функций:
  - Включение множеств:

    $S \subseteq T$ тогда и только тогда, когда формула $\chi_S \Rightarrow \chi_T$ равна 1

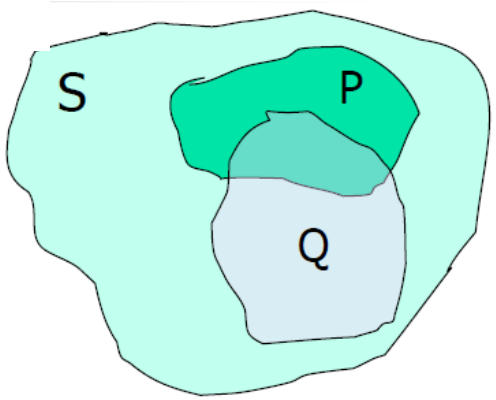
    $S \subset T$ тогда и только тогда, когда формула $(\chi_S \Rightarrow \chi_T) \wedge \neg(\chi_T \Rightarrow \chi_S)$ равна 1
  - Равенство множеств

    $S = T$ тогда и только тогда, когда формула χ_S и χ_T равнозначны.

    Проверить можно по каноническому представлению функций

Для решения этих проблем мы не перебираем элементы множеств, а проверяем, равна ли 1 данная двоичная функция

Операции над множествами и булевы операции (при одинаковой кодировке подмножеств



Нульарные операции (константы):

- полное множество S: $\chi_S \equiv 1$
- пустое множество: $\chi_\varnothing \equiv 0$

Унарная операция:

- дополнение множества S: $\chi_{S \setminus Q} \equiv \neg\chi_Q$

Бинарные операции:

- пересечение множеств: $\chi_{P \cap Q} \equiv \chi_P \wedge \chi_Q$
- объединение множеств: $\chi_{P \cup Q} \equiv \chi_P \vee \chi_Q$
- разность множеств: $\chi_{P-Q} \equiv \chi_P \wedge \neg\chi_Q$

ВСЕ операции над множествами выражаются через булевы операции над характеристическими булевыми функциями

Задача: докажите, что два множества $A \cap (B \cup C)$ и $(A \cap B) \cup (A \cap C)$ совпадают при любых A, B и C с помощью свойств БФ

BDD в обработке данных

- BDD широко используется для эффективного представления конечных структур данных и эффективных алгоритмов, работающих с ними. Три причины использования BDD:
  - компактность представления данных
  - эффективность алгоритмов манипулирования этими данными
  - каноничность представления двоичных функций с помощью BDD
- Можно провести аналогию с использованием позиционной числовой записи вместо унарного кода (в унарной системе счисления количество N представляется N единицами). В обычных алгоритмах проводится перебор элементов конечных множеств по одному. В алгоритмах, основанных на BDD, операции выполняются над МНОЖЕСТВАМИ данных, представленных характеристическими функциями
- Это революция во всех областях, связанных с обработкой дискретной информации. В частности:
  - BDD являются основной моделью для представления и обработки информации в области разработки VLSI
  - использование BDD в ИИ дало новую жизнь многим направлениям в ИИ (например, оказалось очень успешным в задачах планирования)
- Разработаны системы эффективного манипулирования отношениями для решения разнообразных задач на основе BDD: *CrocoPat: A Tool for Simple and Efficient Relational Computation,* http://mtc.epfl.ch/~beyer/CrocoPat/ (2005)



Недостатки представления БФ в форме BDD

- Два недостатка:
  - сложность представления в BDD зависит от порядка переменных. Например, при реализации сложения при одном порядке переменных BDD имеют линейную сложность, при другом – экспоненциальную
  - для некоторых функций (например, двоичное умножение) все функции имеют экспоненциальную сложность. Однако, большинство используемых на практике функций представляются эффективно (линейно)



Заключение

- БФ удобны для решения многих задач, но реально с классическими представлениями БФ можно работать только с 4-6 переменными
- BDD – компактная форма представления полностью определенных БФ в виде направленного графа принятия решений без избыточных вершин и без изоморфных подграфов. Сложность BDD зависит от порядка переменных
- Для большинства БФ представление в BDD линейно или полиномиально. Но существуют классы БФ, для которых представление в BDD экспоненциально
- BDD имеют множество привлекательных свойств:
- каноническое представление
- логические операции с BDD выполняются эффективно
- многие задачи (например, SAT) решаются эффективно
- BDD как форма представления БФ может представлять любые конечные структуры данных (множества, отношения, функции и т.п.) на основе использования булевых характеристических функций (символьное представление).
- Основное свойство BDD – возможность работы с полиномиальным временем на структурах, растущих экспоненциально (например, мощность множества растет экспоненциально, его представление в BDD – линейно)
- Существует много расширений BDD

СПАСИБО
ЗА ВНИМАНИЕ